# REST (Representational State Transfer)

# 1. What is REST?

**REST** is an architectural style for designing web APIs using standard HTTP methods.

It is based on:

```
Resources

Stateless communication

Client-Server model

Uniform interface

Cacheability
```

Used in most modern backend systems.

# 2. Core Idea

Everything is treated as a **resource**.

Examples:

```
None
/users

/products

/orders

/payments
```

Each resource is accessed via URL.

# 3. HTTP Methods in REST

| Method | Purpose |
| --- | --- |
| GET | Read data |
| POST | Create data |
| PUT | Replace full resource |

| | |
|---|---|
| PATCH | Partial update |
| DELETE | Remove resource |

Example:

```
None
GET /users/10

POST /orders

DELETE /cart/55
```

# 4. Stateless Nature

Each request contains all required information.

Server does **not** store client session state between requests.

Example:

```
None
Authorization Token

User ID

Request Params
```

Every request is independent.

# 5. Why REST is Popular

```
Simple to learn

Uses HTTP standards

Easy debugging with curl/Postman

Language independent

Scalable

Works well on internet
```

# 6. Request Example

```
GET /products/101 HTTP/1.1

Host: api.shop.com

Authorization: Bearer token123
```

Response:

```JSON
{
  "id": 101,
  "name": "Laptop",
  "price": 55000
}
```

# 7. JSON in REST

Most REST APIs use JSON.

Example:

```JSON
{
  "name": "Samarth",
  "email": "sam@example.com"
}
```

Easy for frontend + backend.

# 8. Resource Naming Best Practices

Use nouns, not verbs.

Good:

```
None
/users

/orders

/products/10
```

Bad:

```
None
/getUsers

/createOrder

/deleteProduct
```

# 9. REST URL Structure

```
None
GET /users

GET /users/5

GET /users/5/orders

POST /users

PUT /users/5

DELETE /users/5
```

Clean hierarchical design.

# 10. Hypermedia (HATEOAS)

REST can return next possible actions in response.

Example:

```
JSON
{
  "invoice_id": "abc123",
  "links": {
    "pay": "/payments/abc123"
  }
}
```

Meaning:

```
Invoice created

Next action = pay invoice
```

Useful but less common in real systems.

# 11. Caching in REST

REST supports HTTP caching.

Headers:

```
Cache-Control

ETag

Expires

Last-Modified
```

Benefits:

```
Lower latency

Less server load

Higher scalability
```

Example:

```
Cache-Control: max-age=3600
```

# 12. ETag Example

Server sends:

```
ETag: "v7-product101"
```

Client later sends:

```
If-None-Match: "v7-product101"
```

If unchanged:

```
304 Not Modified
```

No full response body needed.

# 13. Status Codes

| Code | Meaning |
| --- | --- |

| | |
|---|---|
| 200 | Success |
| 201 | Created |
| 204 | No content |
| 400 | Bad request |
| 401 | Unauthorized |
| 403 | Forbidden |
| 404 | Not found |
| 500 | Server error |

# 14. REST in Microservices

Used for communication between:

```
None
Frontend ↔ Backend

Mobile App ↔ Backend

Service ↔ Service

Public APIs
```

# 15. Pagination Example

```
None
GET /products?page=2&limit=20
```

Used for large datasets.

# 16. Filtering Example

```
None
GET /products?category=laptop&brand=dell
```

# 17. Versioning

REST has no universal versioning standard.

Common:

```
None
/v1/users

/v2/users
```

Example:

```
None
api.company.com/v1/orders
```

# 18. Security in REST

Usually uses:

```
None
HTTPS

JWT Token

OAuth2

API Keys

Rate Limiting
```

# 19. Advantages

None

Simple architecture

Wide adoption

Easy testing

Supports caching

Scales well

Works across platforms

Readable URLs

# 20. Disadvantages

None

No built-in schema like GraphQL/gRPC

Can over-fetch data

Many round trips possible

Versioning not standardized

Documentation must be added separately

# 21. REST vs RPC vs GraphQL

| Feature | REST | RPC | GraphQL |
|---|---|---|---|
| Style | Resource-based | Function call | Query language |
| Easy to learn | Yes | Medium | Medium |
| Caching | Strong | Weak | Harder |
| Flexible data fetch | Medium | Low | High |

# 22. Interview Use Cases

Use REST when:

```
None
CRUD APIs

Public APIs

Mobile apps

Web apps

Simple microservices

Standard HTTP ecosystem needed
```

# 23. Best Practices

```
Use nouns in URLs

Use correct HTTP verbs

Use status codes properly

Use pagination

Secure with HTTPS

Use caching headers

Version APIs

Provide OpenAPI docs
```

# 24. Real Example

E-commerce:

```
GET /products

GET /products/10

POST /cart

POST /orders

GET /orders/500
```

# 25. One-Line Summary

**REST is a stateless HTTP-based API architecture where resources are exposed through URLs and manipulated using standard HTTP methods like GET, POST, PUT, and DELETE.**